



中华人民共和国国家标准化指导性技术文件

GB/Z XXXXX—XXXX

智能计算 GPU 虚拟化与性能优化技术要求

Intelligent computing—Technical requirements for GPU virtualization and performance optimization

(点击此处添加与国际标准一致性程度的标识)

(本草案完成时间：2026 年 3 月)

在提交反馈意见时，请将您知道的相关专利连同支持性文件一并附上。

XXXX – XX – XX 发布

国家市场监督管理总局
国家标准化管理委员会 发布

目 次

前 言 II

1 范围 1

2 规范性引用文件 1

3 术语和定义 1

4 缩略语 2

5 GPU 虚拟化 2

5.1 概述 2

5.2 硬件层 2

5.2.1 单根输入输出虚拟化 3

5.2.2 多实例 GPU 3

5.2.3 GPU 直通技术 3

5.3 内核层 3

5.3.1 GPU 驱动拦截 3

5.3.2 GPU 驱动半虚拟化 3

5.4 基础支持库层 4

5.4.1 API 拦截 4

5.4.2 API 转发 4

5.5 应用层 5

5.5.1 应用接口 5

5.5.2 资源管理 5

5.5.3 性能监控 6

5.5.4 安全管理 6

5.5.5 弹性调度 6

6 性能优化 6

6.1 概述 6

6.2 单应用性能优化 7

6.3 多应用性能优化 7

6.4 负载变化的多应用性能优化 7

附 录 A 8

前 言

本文件为规范类指导性技术文件。

本文件按照GB/T 1.1—2020《标准化工作导则 第1部分：标准化文件的结构和起草规则》的规定起草。

请注意文件的某些内容可能涉及专利。本文件的发布机构不承担识别专利的责任。

本文件由全国智能计算标准化工作组（SAC/SWG32）提出并归口。

本文件起草单位：中国信息通信研究院、之江实验室、北京趋动智能科技有限公司、天翼云科技有限公司、中国电信集团有限公司、中电信数智科技有限公司、中兴通讯股份有限公司、中移（杭州）信息技术有限公司、中国铁塔股份有限公司、京东科技信息技术有限公司、智己汽车科技有限公司、山东省计算中心（国家超级计算济南中心）、浪潮云信息技术股份公司、中国科学院计算机网络信息中心、北京思谋智能科技有限公司、江苏博云科技股份有限公司、中国移动通信集团有限公司、中移九天人工智能科技（北京）有限公司

本文件主要起草人：庄祎、任垠垠、郑立、韩思齐、栗蔚、陈屹力、李振廷、王鲲、陈飞、江政雄、崔娜妮、蔡旭辉、李响、张子浪、刁雪飞、韩冬、张博、方敏伟、王春晓、郑佳佳、陆腾、张驰、张振华、杨经纬、郑卿、张宇轩、周小淇、王波、朱俊威、苏世鹏、张磊、赵淳、晁元宁。

智能计算 GPU 虚拟化与性能优化技术要求

1 范围

本文件规范了GPU虚拟化与性能优化的技术要求，GPU虚拟化包括硬件层、内核层、基础支持库层、应用层。性能优化方式包括单应用性能优化、多应用性能优化以及负载变化的多应用性能优化。

本文件适用于支持GPU虚拟化相关产品的设计、研发和测试。

2 规范性引用文件

下列文件中的内容通过文中的规范性引用而构成本文件必不可少的条款。其中，注日期的引用文件，仅该日期对应的版本适用于本文件；不注日期的引用文件，其最新版本（包括所有的修改单）适用于本文件。

GB/T 35293-2017 信息技术 云计算 虚拟机管理通用要求

3 术语和定义

GB/T 35293-2017，GB/T 44158-2024界定的以及下列术语和定义适用于本文件。

3.1

虚拟机 virtual machine

一种虚拟的数据处理系统，在某个特定用户的独占使用下，通过共享数据处理系统的各种资源实现其功能。

[来源：GB/T 35293-2017，3.1]

3.2

图执行处理器 graph executor

一种用于管理和执行计算图的系统组件，通常出现在深度学习框架或分布式计算系统中。

3.3

GPU 虚拟化 GPU virtualization

通过软件或硬件技术将物理GPU资源进行抽象、分割与分配，使多个虚拟机或容器能够独立访问和使用GPU资源的技术，实现物理GPU资源的共享与高效利用。

3.4

单根输入输出虚拟化 single root I/O virtualization

一种基于PCIe总线的硬件虚拟化技术，通过在单个物理PCIe设备上创建多个独立的“虚拟功能”，使每个虚拟功能可被虚拟机或容器直接识别为独立的PCIe设备，从而实现单个物理GPU支持多虚拟机/容器并行访问的技术。

3.5

性能优化 performance optimization

通过采用内存管理优化、任务调度算法改进、通信机制升级等技术手段，提升GPU在虚拟化环境下的计算效率、降低资源损耗，以满足不同应用场景性能需求的过程。

3.6

内存管理单元 I/O memory management unit

在硬件与虚拟化软件支持下，将物理GPU直接分配给特定虚拟机或容器，使其绕过虚拟化层直接访问GPU硬件资源，减少虚拟化开销的技术。

4 缩略语

下列缩略语适用于本文件。

- API：应用编程接口（Application Programming Interface）
- DMA：直接内存访问（Direct Memory Access）
- GPU：图像处理单元（Graphics Processing Unit）
- IOMMU：I/O 内存管理单元（Input/Output Memory Management Unit）
- PCIe：外设组件互联标准（Peripheral Component Interconnect Express）
- RDMA：远程直接数据存取（Remote Direct Memory Access）
- SR-IOV：单根输入/输出虚拟化（Single Root Input Output Virtualization）
- TOPS：每秒万亿次操作（Tera Operations Per Second）
- VM：虚拟机（Virtual Machine）

5 GPU 虚拟化

5.1 概述

GPU虚拟化的框架见图1。GPU虚拟化通过硬件辅助与软件协同，在物理GPU硬件层之上构建资源抽象层，对物理GPU的计算核心、显存、带宽等资源进行切片、分配与调度，支持多个虚拟机或容器独立、安全地共享单块或多块物理GPU资源，从而提升物理GPU资源利用率，增强多场景下资源分配的灵活性。GPU虚拟化框架涵盖硬件层、内核层、底层库层及应用层，硬件层提供资源隔离与虚拟化基础能力，内核层负责资源调度与驱动虚拟化，底层库层实现API拦截与转发，应用层提供标准化访问接口。

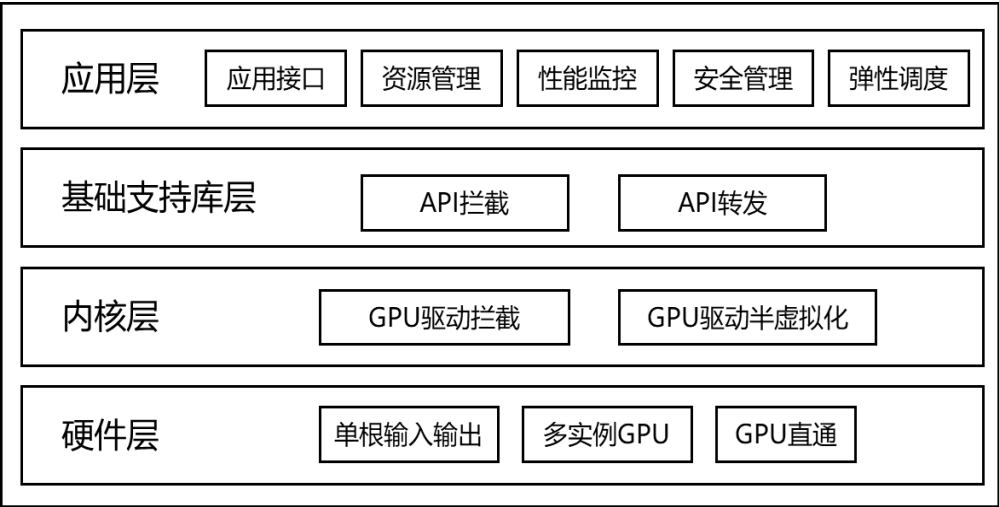


图1 GPU虚拟化框架图

5.2 硬件层

5.2.1 单根输入输出虚拟化

- a) 应全面支持 SR-IOV 技术，通过创建多个虚拟功能，实现一个物理 GPU 被多个虚拟机直接访问。
- b) 虚拟 PCIe 设备的创建和管理应确保设备拥有独立的内存空间、中断和 DMA 数据流，降低 I/O 设备虚拟化开销。

5.2.2 多实例 GPU

- a) 应支持将一块物理 GPU 分割成多个独立的 GPU 实例，每个实例具备独立的 GPU 核心、GPU 显存等资源，可独立运行不同的工作负载；
- b) 应确保虚拟化实例之间的资源独立性和隔离性，在任何负载情况下，实例间的资源干扰率应不超过 1%，提供稳定可靠的性能；
- c) 应支持根据工作负载动态重新配置虚拟化实例资源，包括但不限于 GPU 核心数量、GPU 显存容量、带宽等关键参数，通过资源弹性调整机制提升资源利用效率与任务执行性能；
- d) 应具备完善的虚拟化实例隔离机制，当一个实例出现故障时，故障传播范围应仅限于本实例，不影响其他实例的正常运行；
- e) 应支持云计算、虚拟化环境和容器化环境等多种使用环境，以满足不同的部署需求。

5.2.3 GPU 直通技术

- a) 应全面支持 PCIe 设备的虚拟化，包括 PCIe 总线和驱动的虚拟化，确保设备虚拟化的完整性；
- b) 应兼容主流的 GPU 驱动程序，保障虚拟机对 GPU 硬件资源的正常访问；
- c) 应支持 GPU 卡间互联技术，实现多 GPU 在虚拟机中的协同工作。

5.3 内核层

5.3.1 GPU 驱动拦截

内核拦截模块模拟一个设备文件，内核拦截模块将用户进程在用户态中对该设备文件的访问进行拦截，并调用GPU内核态驱动的函数，然后将函数执行的返回结果进行解析，再返回用户态。

- a) 应支持内核模块创建模拟的设备文件，并输出给用户空间进行使用；
- b) 应支持将模拟的设备文件挂载到容器里，伪装成真实的 GPU 设备文件；
- a) 应支持基础支持库通过设备文件访问 GPU 驱动的功能；
- b) 应支持对特定的设备访问接口进行功能模拟，支持不同应用程序在使用 GPU 显存、GPU 算力时相互隔离。
- c) 应支持故障隔离，应用程序在使用模拟的设备文件过程中任何形式的异常，均不影响其他正在使用相同物理设备的应用程序正常运行。
- d) 应支持防虚拟化逃逸，在容器中的应用程序不应该具备绕过模拟设备而直接使用物理 GPU 的能力。
- e) 访问伪装的设备文件，所有访问请求被内核模块拦截，内核模块根据请求协议内容按需对请求进行调度与准入控制，再将拦截的访问请求转发给真实的 GPU 驱动，解析返回结果，并返回给用户态应用程序

5.3.2 GPU 驱动半虚拟化

使用定制化的GPU驱动，用户进程通过该定制化GPU驱动提供的虚拟化接口，访问物理GPU的功能。

- a) 应支持未修改的用户态应用程序和基础支持库在虚拟机内运行；

- b) 应支持应用程序使用虚拟 GPU 时的服务质量保证，确保不同虚拟机或应用程序的 GPU 请求能够按照优先级和策略得到合理的服务。
- c) 应支持不同应用程序在使用 GPU 显存、GPU 算力时相互隔离。
- d) 应支持故障隔离，应用程序以及虚拟机在使用虚拟 GPU 过程中任何形式的异常，均不影响其他正在使用相同物理设备的应用程序正常运行。
- e) 应支持防虚拟化逃逸，在虚拟机中的应用程序以及定制化的 GPU 驱动均不应该具备绕过虚拟化机制而直接使用物理 GPU 的能力。

5.4 基础支持库层

5.4.1 API 拦截

API拦截是指在应用程序发起对基础支撑库的API调用过程中，通过特定机制捕获并接管该调用请求，暂停其原生执行流程，以实现调用行为的监控、改写或转发等管控操作。

- a) 应支持在用户态实现一个或多个函数库，该函数库应复现基础支持库的所有 API 接口；
- b) 应支持应用程序通过函数库间接调用基础支持库的 API；
- c) 应支持函数库拦截应用程序的函数调用，对函数调用进行解析，并使用解析获得的参数去调用实际的基础支持库中相同能力的 API；
- d) 应支持函数调用完成后，函数库将结果返回给应用程序；
- e) 应支持函数库能够与基础支持库的版本兼容，当基础支持库的版本升级时，函数库也能够自动适配新的版本，确保应用程序的正常运行；
- f) 应支持函数库在拦截函数调用前，对传入的参数进行校验和预处理，以确保参数的合法性和有效性，避免潜在的安全风险；
- g) 应支持函数库在拦截函数调用后，对返回结果进行处理和过滤，确保返回给应用程序的结果符合预期；
- h) 应支持函数库可以根据需要动态替换对基础支持库 API 调用的实现，从而实现对特定功能的扩展；
- i) 应支持函数库在拦截函数调用前，应根据函数类型对函数调用进行调度与准入控制，从而实现算力和显存隔离控制。

5.4.2 API 转发

API转发是指在拦截应用程序对基础支撑库的 API 调用后，将该API的实际执行逻辑转发至本地节点或其他节点的 GPU 上完成后续处理。

- a) 应支持函数库通过网络，跨越不同的操作系统实例远程调用其他机器上的基础支持库的 API；
- b) 应支持函数库分为两部分，客户端负责 API 拦截之后的转发请求，服务器端负责接收请求并调用相应的基础支持库的 API；
- c) 应支持通过该方式实现 GPU 资源的池化管理，即多个 GPU 可以组成 GPU 资源池，由多个客户端，以支持不具备 GPU 硬件的机器也能通过调用 GPU 资源池，实现 GPU 加速的功能；
- d) 应支持跨越虚拟机、容器和裸金属三种云基础架构下的 GPU 虚拟化和池化，且三种架构下能力一致；
- e) 应支持函数库能够与现有的分布式计算框架和资源管理系统集成；

- f) 宜支持函数库在网络通信中实现数据加密和认证机制，确保 API 转发过程中的数据安全和隐私保护；
- g) 应支持函数库在 API 转发过程中，保持应用所在的业务环境和物理 GPU 资源所在运维环境的网络隔离；
- h) 应支持函数库在 API 转发过程中具备容错和负载均衡能力，当服务器端出现故障或负载过高时，能够切换到其他健康的服务器，以保证服务的可用性；
- i) 应支持函数库能够对 API 转发过程进行监控和统计，收集如响应时间、吞吐量等性能指标，以便对系统进行优化和故障排查；
- j) 应支持函数库能够实现 API 转发和调用的版本控制，确保客户端和服务端之间的接口一致性，避免因版本不匹配导致的调用失败。
- k) 应支持支持跨节点的 GPU 资源聚合，也即是支持 API 同时转发至多台 GPU 物理服务器，为单一容器、虚拟机和裸金属环境提供来自不同节点的多卡 GPU 资源，并对业务应用保持和使用本节点多卡资源一致的使用方式；
- l) 应支持 GPU 虚拟资源动态调整，无须重启容器、虚拟机和裸金属等运行环境；
- m) 应支持客户端和服务端的热升级，升级时不影响正在运行中的应用程序。

5.5 应用层

5.5.1 应用接口

应用接口负责提供标准化的编程接口，使得开发者可以方便地使用 GPU 资源。主要包括：

- a) 应支持标准化接口，提供统一的 API 标准，使得不同虚拟化平台上的应用可以无缝迁移；应兼容主流的 GPU 硬件及通用并行计算架构；应支持跨平台 API 封装，使得上层应用不依赖底层 GPU 硬件或虚拟化实现；应提供 API 版本兼容机制，支持向后兼容，避免因接口升级导致现有应用不可用；接口应与业务系统解耦，无需依赖数据库、用户组织结构等；
- b) 应支持 SDK，提供开发工具包，帮助开发者快速集成和使用 GPU 虚拟化功能。应支持多种编程语言；应提供开发文档与示例代码，帮助开发者理解接口使用方式，包含典型场景示例；
- c) 应支持主流 AI 框架及图形渲染框架；
- d) 宜支持多编程语言接口，提升跨生态开发与兼容能力；
- e) 宜提供应用层服务质量控制接口。

5.5.2 资源管理

资源管理主要关注如何高效地分配和使用 GPU 资源，以满足多用户的需求。主要包括：

- a) 应支持显存隔离，对 GPU 显存的精细化切分与隔离，确保不同应用、虚拟机或容器拥有独立的显存空间，避免因显存共享导致的数据泄漏、冲突或性能干扰。
- b) 应支持算力隔离，在 GPU 虚拟化环境下，对计算核心进行资源划分与隔离，在不同优先级任务的并发运行、异构算力单元的动态切分、异构芯片集群的统一算力管理等场景下保证多任务并行运行时互不干扰。通过多任务并发压力测试，借助算力分配精度等指标，验证算力隔离效果与调度公平性；
- c) 应支持故障隔离，实现快速识别和隔离故障，保证系统稳定性；
- d) 应具备快速识别并隔离 GPU 节点或任务故障的能力，避免单点故障扩散导致系统整体崩溃，降低单 GPU 故障对多任务运行的影响、集群内节点宕机的波及范围以及驱动/运行时错误导致的异常传播；

- e) 应支持对多媒体编解码单元的隔离，支持视频转码与 AI 视频分析混合任务、高并发流媒体推理、不同租户的视频任务同时运行等环境，避免视频流任务或 AI 视频推理任务间产生带宽争用和性能互扰；
- f) 应支持资源回收，及时释放闲置资源，提高资源利用率；
- g) 应支持动态资源调整，根据应用负载实时调整 GPU 资源分配，支持添加和删除 GPU 服务器节点或 GPU 卡；
- h) 宜支持显存超分，以提高资源利用效率。

5.5.3 性能监控

性能监控主要关注实时监控GPU资源的使用情况，并提供性能优化建议。主要包括：

- a) 应支持资源使用监控，包括显存占用、计算资源利用率、编解码器利用率、显存带宽等等；
- b) 应支持性能瓶颈分析，识别和分析系统中的性能瓶颈，提供优化建议；
- c) 应支持日志记录，帮助排查故障和进行性能调优；
- d) 应支持性能预警，当资源使用超过阈值时发出告警；
- e) 应支持多维度监控，包括容器级和虚拟机级的监控。

5.5.4 安全管理

安全管理主要关注在虚拟化环境中确保GPU资源的安全使用。包括：

- a) 应支持访问控制，基于角色的访问控制，确保只有授权用户才能访问和使用 GPU 资源；
- b) 应支持数据加密数据传输和存储过程中，使用加密技术保护敏感数据；
- c) 应支持安全事件检测，监测和检测潜在的安全事件和异常行为，以保护 GPU 资源免受未经授权的访问和攻击；
- d) 应支持 GPU 卡上计算任务的热迁移能力，将任务热迁移到其它服务器上剩余资源充足的 GPU 卡上，任务能够继续运行而上层业务无感知；
- e) 应支持在业务不停机的状态下，进行 GPU 虚拟化相关组件的灰度升级部署；
- f) 应支持安全审计，记录所有关键操作，便于事后审计和追溯；
- g) 应支持故障自动迁移，当检测到 GPU 节点故障时，自动将任务迁移到健康节点；
- h) 宜支持硬件辅助安全，利用硬件虚拟化技术实现设备直通与内存隔离，防止侧信道攻击。

5.5.5 弹性调度

弹性调度主要关注根据任务需求动态分配和调整GPU资源，以提高资源利用率和任务执行效率。包括：

- a) 应支持任务队列管理，合理调度和分配 GPU 资源；
- b) 应支持优先级调度，确保高优先级任务优先获得资源；
- c) 应支持抢占式调度，允许高优先级任务抢占低优先级任务的资源；
- d) 应支持负载均衡，将任务均匀分配到多个 GPU 上，避免单点过载；
- e) 应支持弹性扩缩容，根据负载动态增加或减少 GPU 资源。

6 性能优化

6.1 概述

GPU虚拟化性能优化方式主要有三种，分别为单应用性能优化、多应用性能优化以及负载变化的多应用性能优化。

6.2 单应用性能优化

在虚拟化、资源池化的条件下，对单个任务的性能进行优化。

- a) 宜通过优化应用的显存布局、API 调用，提高任务的性能；
- b) 应通过优化分布式任务的拓扑布局，提高分布式任务的性能；
- c) 宜通过利用系统的高性能网络，提高分布式任务的网络性能；
- d) 应通过 RDMA 实现跨节点 GPU 直接通信，降低分布式训练中的数据传输延迟。

6.3 多应用性能优化

通过合理的算力池化方案和资源调度，当一个集群同时运行多个独立应用的时候，对整个集群的系统吞吐能力进行优化。

- a) 应利用不同应用的负载特性，自动合理匹配不同型号的算力硬件，减少算力资源浪费，提高整体的系统吞吐；
- b) 在多应用调度时，调度器应合理匹配应用关于 CPU、内存、智能算力、存储、网络的多维度需求，应通过算力池化、存储池化的方案解耦合各种需求，使得多个应用能够充分使用集群内各种资源，提高系统吞吐。

6.4 负载变化的多应用性能优化

当应用的负载随时间呈现波动的时候，通过合理的迁移和调度，合并资源需求，在相同的标称硬件算力条件下支持更多的应用，提高整个集群的吞吐

- a) 应在业务负载减轻的时候缩小甚至完全释放 GPU 资源；
- b) 合理预估多个应用的业务负载变化，并在检测到负载减轻的时候，应通过热迁移的能力重新均衡 GPU 算力分布，合并小负载应用，释放出更多完整的 GPU 资源用以支持更多的应用；
- c) 多个具有负载冷热变化的应用，宜通过快速资源切换技术使得多个应用有效共享单组 GPU 资源。

附录 A

(资料性)

性能评估指标要求

性能指标评估主要用于衡量 GPU 虚拟化系统在实际应用中的表现，通过多个维度进行全面分析和评估。

1 弹性度

弹性度主要评估系统在不同负载和不同业务场景下的动态分配、调整和伸缩资源的能力。高弹性的虚拟化系统不仅能够根据需求动态分配资源，还能够任务运行过程中，根据负载变化调整资源，从而兼顾任务的高效执行和资源的充分利用。

应根据业务场景对弹性度的不同要求，弹性度评估可划分为场景驱动的资源切分粒度和多任务并发时的资源调度两个维度。

1.1 场景驱动的资源切分粒度

根据任务的规模、性质和需求，虚拟化系统应当能够在以下场景下进行合理的资源切分：

- a) 大规模任务评估：评估虚拟化系统承载大规模、长周期训练任务的能力，重点考察其动态资源调度与线性扩展效率。通过监控验证系统能否平滑地分配与扩展 GPU 资源，以匹配训练任务在不同阶段的需求，从而避免性能瓶颈并确保训练的高效进行；
- b) 小规模任务评估：评估检验虚拟化系统在处理高并发推理负载时的敏捷性与灵活性。系统应根据推理负载的波动，快速响应并调整资源分配的粒度，在流量洪峰时以极短的冷启动时间细粒度快速扩容以保障服务质量，并在流量回落后迅速、可靠地回收资源以控制成本。

1.2 多任务并发时的资源调度

在多任务并发的场景下，虚拟化系统需要高效地调度和分配 GPU 资源，确保多个任务能够共享资源而不影响彼此的性能。评估内容包括：

- a) 应通过模拟混合负载，重点评估系统能否有效执行优先级策略，在保障关键任务性能的同时，实现对资源的公平、高效分配；
- b) 使用监控工具将量化其在高并发下的调度响应时间和资源隔离效果，以确保系统在复杂场景下依然稳健可靠。

2 隔离性

隔离性主要评估系统在资源、性能和安全方面的隔离能力，确保不同虚拟机或容器之间的运行互不干扰。

2.1 资源隔离

资源隔离方面，主要评估 GPU 资源的隔离效果，确保不同用户或任务之间的资源使用互不干扰，避免不同虚拟机之间因共享或冲突而导致的性能下降。可通过资源访问测试和负载测试来验证隔离效果，评估内容包括：

- a) 在多个虚拟机上同时运行任务，监控其 GPU 利用率、内存占用和计算资源分配情况，确保各虚拟机的资源隔离性良好，避免资源竞争或冲突，保证每个虚拟机按预期独立运行；

- b) 考察系统是否支持细粒度的资源划分，例如按 GPU 核心、显存、带宽等进行分配，从而保证多租户环境下的公平性和稳定性；
- c) 结合 MLPerf 等性能基准工具，对多任务场景中的资源隔离能力进行量化测试，检验虚拟化系统是否能在复杂负载下维持一致的性能表现。

2.2 性能隔离

性能隔离主要评估在共享GPU资源的情况下，不同用户或者任务在高负载下仍能保持稳定的性能，不受其他用户或者任务的影响。评估内容包括：

- a) 通过分配高负载任务到不同的虚拟机上，观察其计算能力和 GPU 内存使用情况是否会互相影响；
- b) 通过评估任务的执行时间和资源占用，确保系统能够合理分配和隔离计算资源，即便是在高负载的情况下。

2.3 安全隔离

安全隔离主要评估虚拟化系统是否通过访问控制、数据加密和身份验证来保证不同虚拟机或任务间的数据安全。评估内容包括：

- a) 测试不同虚拟机间的数据访问权限，确保在无权访问的情况下，数据得到有效保护；
- b) 验证平台提供的虚拟化技术是否能够有效隔离 GPU 资源，避免跨虚拟机的隐性访问和潜在的数据泄露；
- c) 通过设置不同的访问控制策略，测试虚拟机或容器是否能够正确隔离和限制不同用户的数据访问；
- d) 验证虚拟化系统是否支持身份验证机制，并进行安全审计，保访问日志和异常行为能被有效记录和监控，防止潜在的安全漏洞。

3 算力折损率

算力折损主要用于衡量虚拟化、资源池化调度、跨设备协同等带来的性能损耗，算力折损率需要基于统一算力标识体系与抽象池化建模来定义和评估，从而指导跨厂商、跨架构的资源分配与系统优化。

3.1 分层算力折算率

算力折损率评估的核心前提，是依托统一标识体系开展分层对比分析，评估维度划分为以下三级：

- a) 节点级折损率针对多接入智算中心节点开展算力折损率评估，重点评估多维资源标识体系在减少节点资源闲置与碎片化、提升算力实际利用率过程中的支撑作用；
- b) 池化级折损率：基于硬件抽象池化技术开展资源利用率评估，重点评估虚拟化与资源切分精度，以及不同任务类型场景下的资源调度效率；
- c) 系统级折损率：通过大规模混合任务试验，量化跨设备协同调度、数据通信及负载均衡所产生的性能损耗，结合监测数据，评估算力折损率随系统规模与任务混合度变化的演化规律。

3.2 度量指标定义

度量指标包括理论峰值算力、实际有效算力：

- a) 理论峰值算力：表示由算力标识体系计算得到的系统在理想/原生情况下的理论峰值计算能力，即在统一标识下算力能力值的总和，或根据任务理想映射得到的“任务理论峰值”；

- b) 实际有效算力：通过硬件虚拟化、抽象池化、资源分配和异构调度后，实际在全局资源池中观测到的最大有效计算能力（或应用级有效吞吐），包含运行时的通信、调度和虚拟化损耗。算力折损率计算公式：

$$\text{算力折损率} = (1 - \frac{P_v}{P_n}) \times 100\%$$

其中：

P_v 表示虚拟后GPU的最大计算性能。

P_n 表示原始GPU的理论峰值计算性能。
